



More Interpretable Decision Trees: Pruning via Node Descent and the Delta Penalty

Miguel Á. Carreira-Perpiñán and Suryabhan Singh Hada,
Dept. CSE, UC Merced

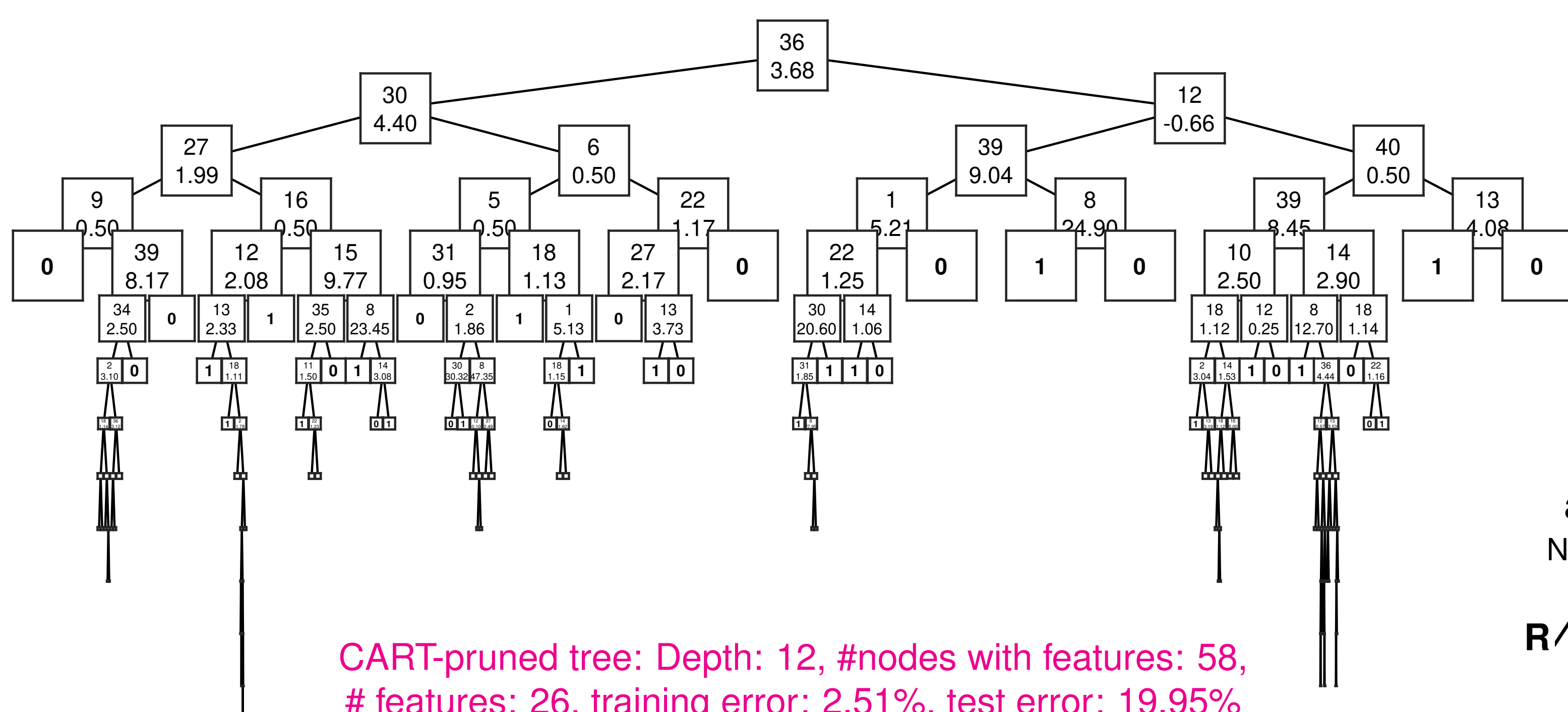
Summary

- Node descent (ND) defines the problem of axis-aligned decision tree (T) pruning as a joint optimization of all its parameters (feature and bias at each decision node, class label at each leaf) of a loss (such as the 0/1 loss) plus a penalty proportional to the number of nodes.
- ND is an iterative algorithm that is guaranteed to decrease or leave unchanged the objective function and stop after a finite number of iterations, in polynomial time.
- It defines the following loss directly in terms of the tree parameters (Θ) (given the initial tree structure) and the number of nodes ($\mathcal{D} \cup \mathcal{L}$) via the delta penalty ($\delta(\cdot)$), then applies coordinate descent one node at a time to optimize the objective function.

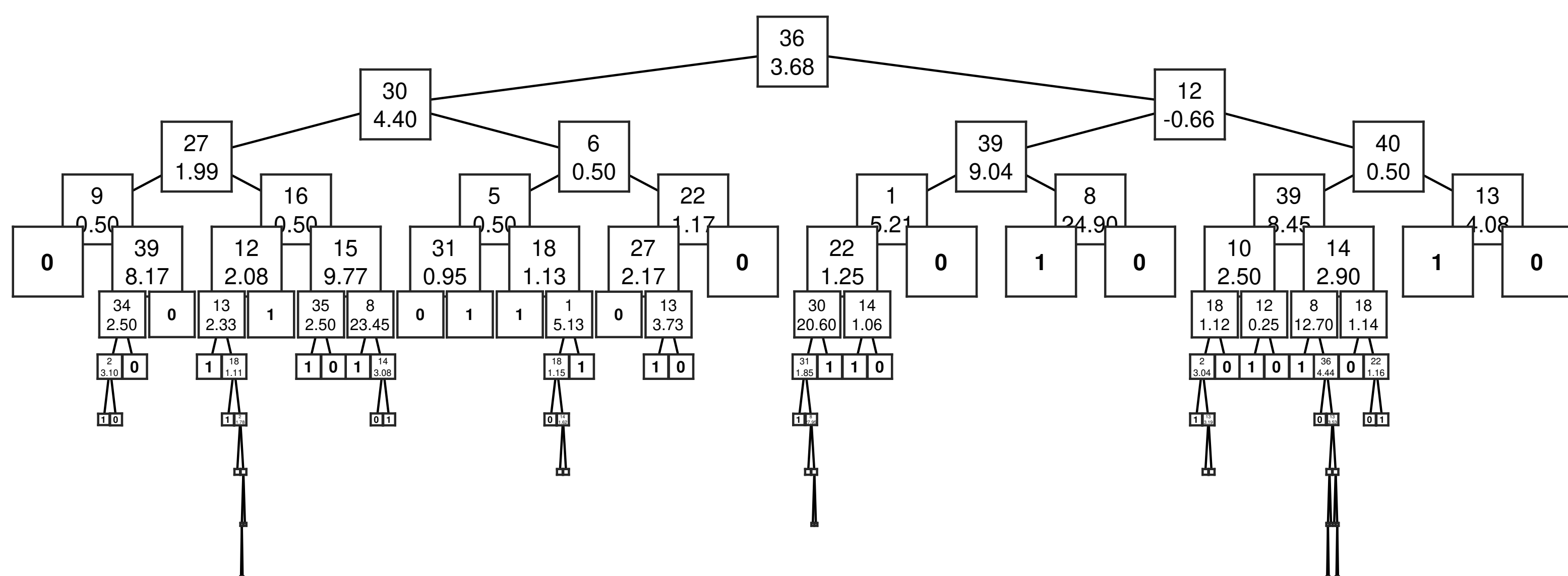
$$\min E(\Theta) = \sum_{n=1}^N L(y_n, T(\mathbf{x}_n; \Theta)) + \lambda \sum_{i \in \mathcal{D}} \delta(w_i)$$

s.t. $w_i \in \{\pm 1, 0\}$, $b_i \in \mathbb{R}$, $i \in \mathcal{D}$; $c_j \in \{1, \dots, K\}$, $j \in \mathcal{L}$

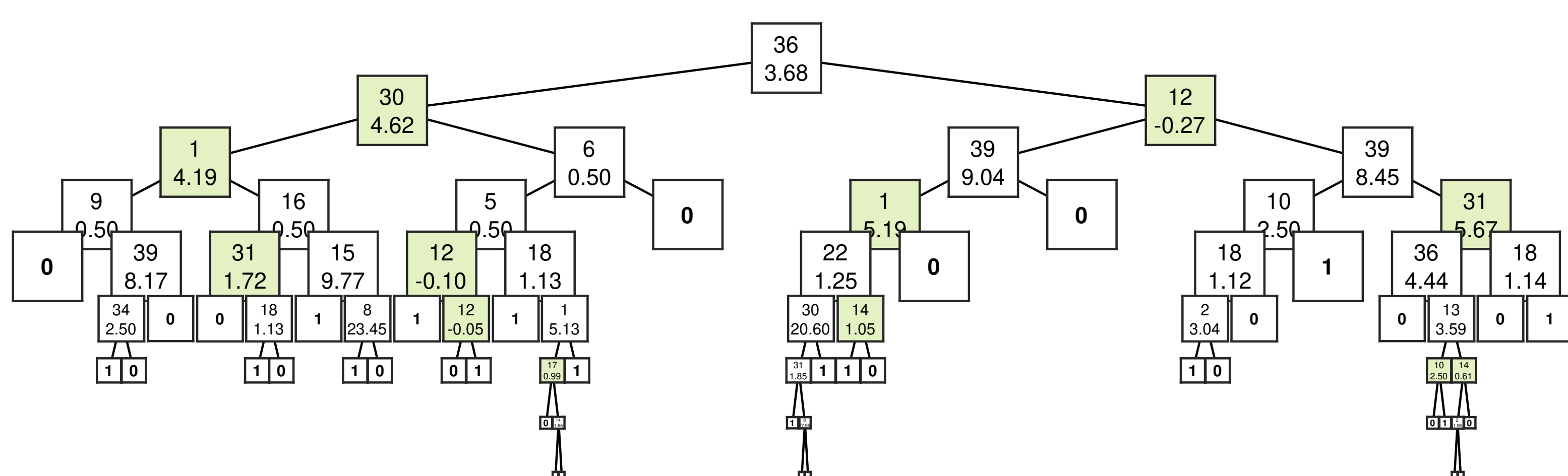
Fully-grown tree: Depth: 13, #nodes with features: 92,
features: 27, training error: 0.00%, test error: 24.22%



CART-pruned tree: Depth: 12, #nodes with features: 58,
features: 26, training error: 2.51%, test error: 19.95%



ND-pruned tree: Depth: 7, #nodes with features: 38
features: 20, training error: 5.36%, test error: 17.86%



Pictorial representation of the result of applying CART pruning or ND to the fully-grown tree for an instance of QSAR dataset ($D = 42$, $K = 2$). Green nodes show changes in node parameters.

```

input binary axis-aligned tree  $T$  with given structure and parameters  $\Theta$ 
  at the nodes  $\mathcal{N} = \mathcal{D} \cup \mathcal{L}$ 
repeat
  for  $i \in \mathcal{N}$ 
     $\mathcal{R}_i \leftarrow$  reduced set of node  $i$ 
  for  $i \in \mathcal{N}$  in reverse BFS order
    if  $\mathcal{R}_i \neq \emptyset$  then
      if  $i \in \mathcal{L}$  then
         $c_i \leftarrow$  majority class in  $\mathcal{R}_i$ 
      else
         $w_i, b_i \leftarrow$  solution of the RP over decision node  $i$ 
until  $E(\Theta)$  does not strictly decrease postprocess  $T$ : remove redundant nodes
return  $T$ 

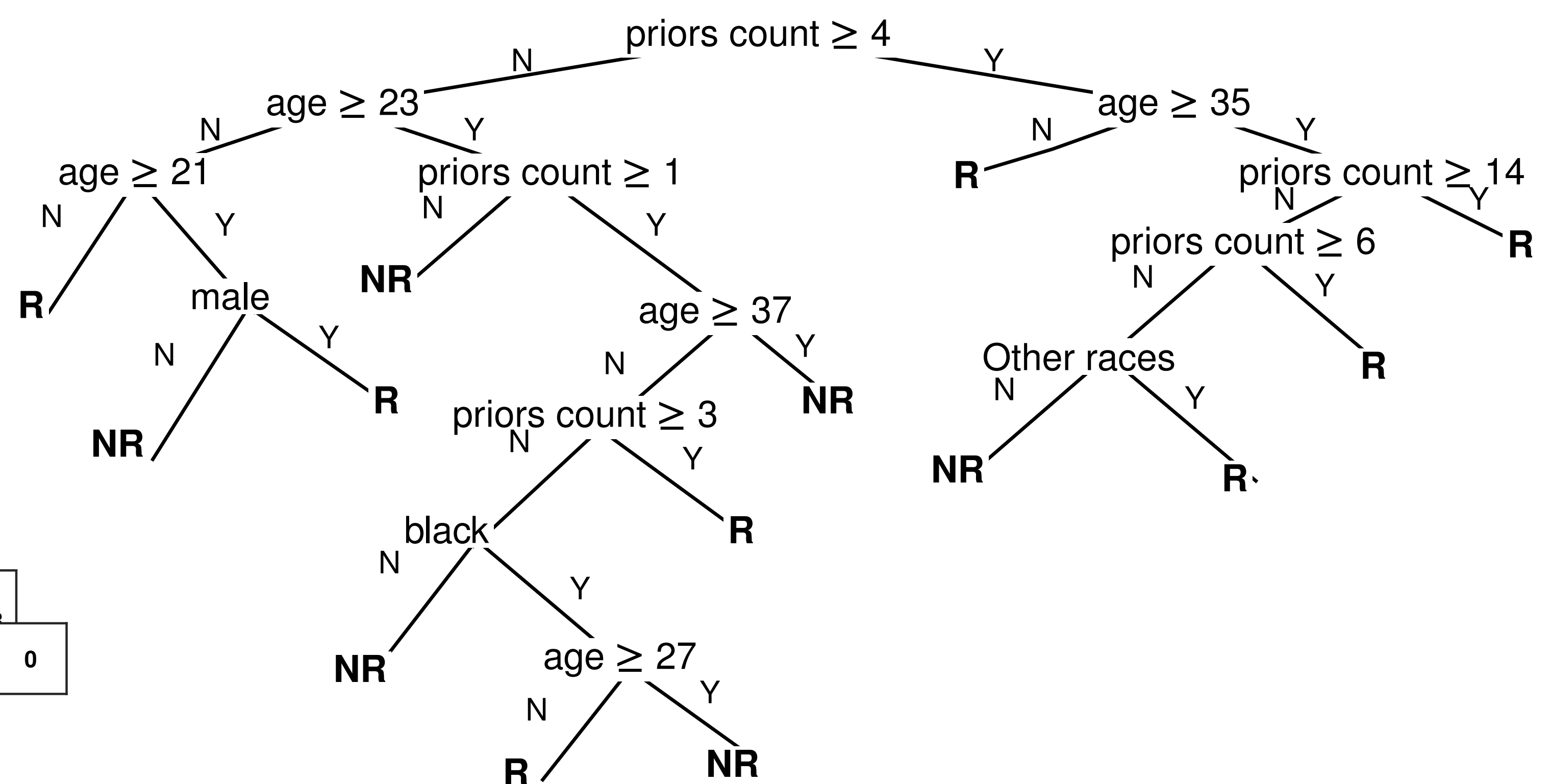
```

Pseudocode for the node descent algorithm.

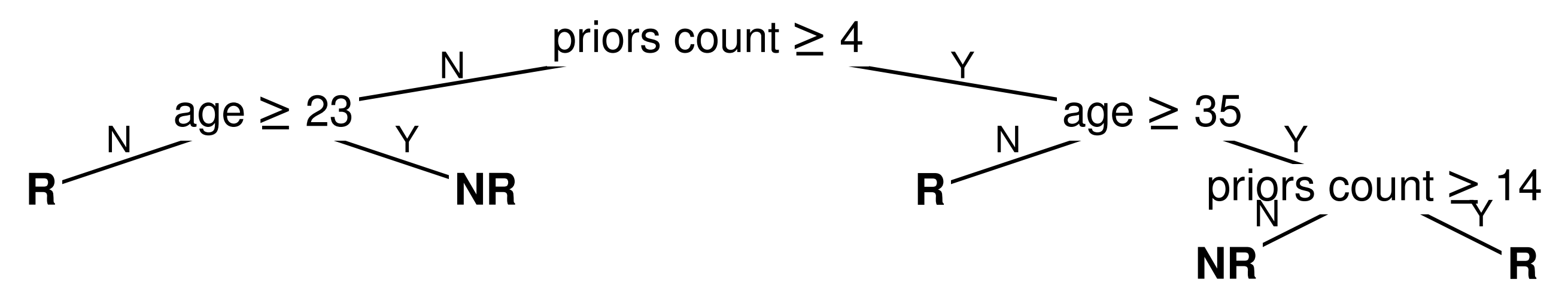
Dataset	(N_{train} , D , K)	Original	CART	TAO	ND
Yeast	(1162, 8, 10)	50.41±2.80	45.02±3.07	44.02±3.34	43.58±2.31
Waveform21	(4000, 21, 3)	23.70±0.59	22.50±1.18	22.50±1.36	21.60±0.91
Car Eval	(1382, 21, 4)	2.55±0.59	2.22±0.59	2.17±0.49	1.97±0.59
Banknote auth	(1078, 4, 2)	1.89±0.26	1.85±0.23	1.78±0.36	1.59±0.50
Sensor	(46808, 48, 11)	1.78±0.09	1.69±0.12	1.50±0.07	1.40±0.15
Connect4	(54045, 126, 3)	24.58±0.04	24.53±0.02	24.66±0.17	21.83±0.09

Test errors for different algorithms in different datasets.

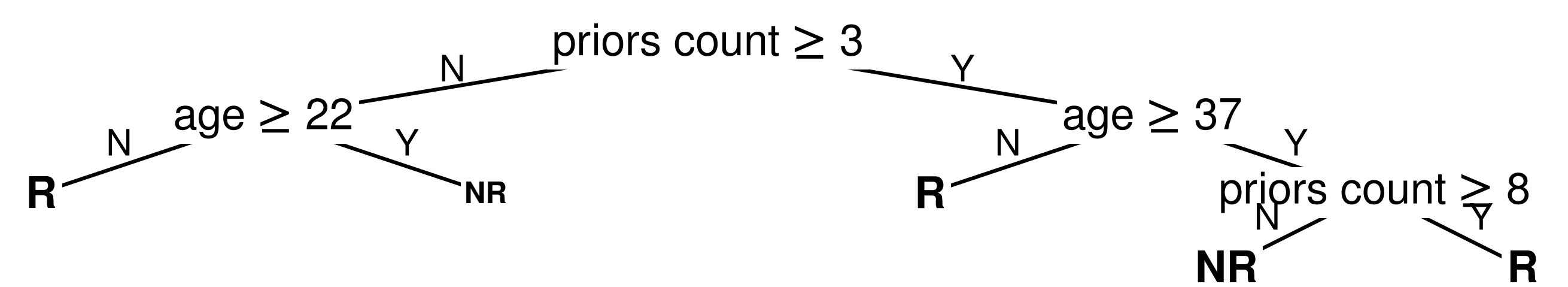
CART-pruned tree: training error: 28.93%, test error: 32.13%, depth: 7,
nodes with features: 13, # features used: 5



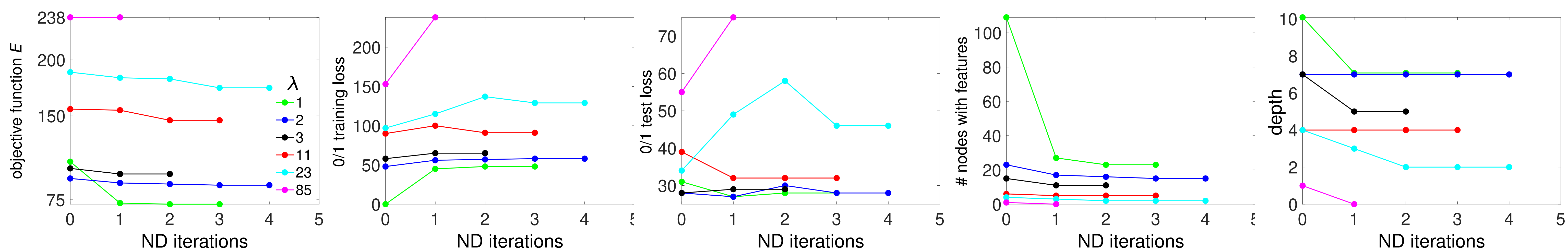
CART-overpruned tree: training error: 30.32%, test error: 34.59% depth: 3,
nodes with features: 4, # features used: 2



ND tree: training error: 29.29%, test error: 30.64%, depth: 3,
nodes with features: 4, # features used: 2



Trees for the COMPAS dataset.



Learning curves of the ND algorithm (Balance Scale dataset).